

InkClear: Project Run and Implementation Report

Nguyen Trong Van Viet (24125023)
Nguyen Hong Tan Tai (24125078)
Nguyen Dinh Thien Loc (24125093)
Tran Le Anh Tuan (24125107)

26 July 2026

Contents

1	Introduction	2
2	Background	2
3	How to run	2
4	Technical Approach	2
5	Enhancement pipeline	3
6	Demo or Case Study	3
7	Evaluation	3
8	Known issues and portability	3
9	AI Usage Statement	4
10	References	4
11	Conclusion	4

1 Introduction

InkClear is an Android application that improves the readability of handwritten document photographs. A user can import an image or scan a single page with Google ML Kit Document Scanner, choose an enhancement mode, tune its strength, compare the source and result, and export the enhanced page as JPEG or PDF. Image processing is performed locally in the app.

2 Background

The repository is a single-module Gradle Android project. The root includes the Gradle wrapper, while the application code and resources are under `app/`. The relevant configuration is summarized below.

Item	Value
Project name	<code>Android_AI_Integration</code>
Application label	InkClear
Application ID	<code>mobile_app.android_ai_integration</code>
Minimum SDK	API 25
Target SDK	API 36
Compile SDK	API 37.1
Gradle wrapper	9.6.1
Android Gradle Plugin	9.3.0
Java compatibility	11
Primary external feature	ML Kit Document Scanner 16.0.0

Table 1: Verified project configuration.

The project uses AndroidX activity, lifecycle, Compose BOM/material dependencies, although the principal screens are XML layouts backed by Java activities. Dependency versions are centralized in `gradle/libs.versions.toml`.

3 How to run

Install Android Studio with SDK Platform 37.1 and use JDK 11 or newer. An emulator or physical device running API 25 or newer is required. The first build needs network access to download the Gradle distribution and Maven dependencies.

From PowerShell at the repository root, build and install the debug application as follows:

Listing 1: Windows build commands

```
.\gradlew.bat assembleDebug
.\gradlew.bat installDebug
```

The APK is produced at `app/build/outputs/apk/debug/app-debug.apk`. In Android Studio, open the repository, let Gradle sync complete, select the `app` configuration, and run it on an API 25+ target. macOS and Linux use `./gradlew` in place of `gradlew.bat`.

4 Technical Approach

The app is organized around a launcher activity, an editing activity, XML layouts, and a dedicated image-enhancement utility. The build uses Gradle and AndroidX libraries, with ML Kit providing optional document scanning.

1. **HomePage** presents the InkClear introduction and opens the editor.
2. **InkClearActivity** accepts an image from the Android document picker or launches ML Kit's scanner for one page.
3. The selected image is decoded, EXIF rotation is applied on older Android versions, and large images are limited to a maximum dimension of 1800 pixels.
4. The enhanced image is shown in a preview. The user can switch between original and enhanced views, select a visual mode, and change strength.
5. The Android document-creation picker saves the result as JPEG or as a one-page PDF.

5 Enhancement pipeline

HandwritingEnhancer converts the source pixels to luminance, estimates the local page background with a separable box blur, and normalizes luminance against that background. Strength controls the ink boost and, for Black and White mode, the binary threshold. Natural mode restores a portion of the original chroma; Grayscale produces equal RGB channels; Black and White emits only black or white pixels.

Processing runs on a single background executor so image decoding and enhancement do not block the UI thread. A generation counter prevents an older asynchronous load from replacing a newer selection. Bitmaps are recycled when replaced to limit memory pressure.

6 Demo or Case Study

The repository includes one local example unit test and one instrumented example test. The local test checks the template arithmetic example. The instrumented test currently asserts the template package name `com.example.android_ai_integration`, while the actual manifest and Gradle configuration use `mobile_app.android_ai_integration`. Therefore the connected test requires that assertion to be corrected before it can be considered a valid project test.

Recommended commands are:

Listing 2: Test commands

```
.\gradlew.bat test
.\gradlew.bat connectedDebugAndroidTest
```

7 Evaluation

The scanner is configured for one page and JPEG results. ML Kit availability can vary by device or emulator, so image import is the fallback path. The app does not declare storage permission; it relies on the Android picker and persistable URI access where provided. Large images are downscaled for predictable processing cost. The current test suite is scaffolding rather than feature coverage, and no automated image-quality benchmark is included.

8 Known issues and portability

The project is portable across computers provided that Android Studio, a compatible JDK, Android SDK Platform 37, an API 25 or newer device/emulator, and network access for the first dependency download are available. The Gradle wrapper supplies the Gradle version, so a

global Gradle installation is unnecessary. Android Studio’s Embedded JDK 17 or 21 is suitable for the Gradle JDK setting.

The following findings were verified during execution:

- The connected example test previously contained a stale template package assertion. It now uses the built application ID `mobile_app.android_ai_integration`, and the connected test passed on the tested emulator.
- `installDebug` installs but does not launch the application. The launcher can be started explicitly with `adb shell am start -n mobile_app.android_integration/.HomePage`.
- The native-library strip message for `libandroidx.graphics.path.so` is a non-fatal packaging warning; the debug APK installed successfully.
- ML Kit scanning depends on device/emulator support. The Android document picker remains the fallback input path.

With the corrected assertion, another computer should be able to run the project with `gradlew.bat installDebug` on Windows or `./gradlew installDebug` on macOS/Linux, assuming the required SDK components and an attached device are available.

9 AI Usage Statement

AI tools supported repository inspection, run-instruction drafting, report composition, and troubleshooting of the Gradle and emulator workflow. Their outputs were verified against the project source files, Gradle configuration, official Gradle checksum data, successful Gradle tasks, emulator installation and launch, connected Android tests, and a successful `latexmk` compilation. No AI-generated code was accepted without checking it against the implementation and observed build behavior.

10 References

- Android Developers, [Android Studio documentation](#).
- Android Developers, [Build your app with Gradle](#).
- Google Developers, [ML Kit Document Scanner](#).
- Gradle, [Gradle Wrapper documentation and release checksums](#).

11 Conclusion

InkClear is ready to build and run as a small, self-contained Android prototype. The README now records the verified prerequisites, commands, user workflow, project layout, and report compilation command. The next useful engineering step is to replace the example tests with tests for enhancement modes, strength behavior, export, and the corrected application package.